

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ – VIỆN THÔNG



HCMUS
Viet Nam National University
Ho Chi Minh City
University of Science

BÁO CÁO LAB 6

MÔN: THỰC HÀNH GIAO TIẾP MÁY TÍNH VÀ THU NHẬN DỮ LIỆU

TÊN ĐỀ TÀI: BLUETOOTH LOW ENERGY - MESH

Lớp: 22DTV_CLC1

Giảng viên hướng dẫn:

- ThS. Đặng Tấn Phát
- CN. Hồ Thanh Bảo

Sinh viên thực hiện:

Nhóm:

- Họ và tên: Lương Hải Long
- Mã số sinh viên: 22207056

THÀNH PHỐ HỒ CHÍ MINH - THÁNG 12 NĂM 2025

MỤC LỤC

MỤC LỤC	2
DANH MỤC HÌNH ẢNH	3
BÀI 1: TRUYỀN TẢI DỮ LIỆU ĐỊNH DANH (MSSV) QUA MẠNG MESH	4
BÀI 2: ỨNG DỤNG MESH - TRUYỀN TẢI TRẠNG THÁI LED VÀ THỜI GIAN HOẠT ĐỘNG .	8

DANH MỤC HÌNH ẢNH

Hình 1 Console của Server - Hệ thống nhận và giải mã thành công MSSV (22207056) từ Client	7
Hình 2 Server - Màn hình LCD hiển thị thời gian hoạt động nhận được từ Client	11
Hình 3 Server - Màn hình hiển thị trạng thái hai LED đều tắt (L0: Off, L1: Off)	11
Hình 4 Server cập nhật trạng thái LED 1 bật (L0: Off, L1: On)	12
Hình 5 Server cập nhật trạng thái LED 0 bật (L0: On, L1: Off)	12
Hình 6 Server hiển thị cả hai LED đều bật (L0: On, L1: On)	13

BÀI 1: TRUYỀN TẢI DỮ LIỆU ĐỊNH DANH (MSSV) QUA MẠNG MESH

1. Yêu cầu bài toán

- Thiết lập hệ thống Bluetooth Mesh với mô hình Client-Server để thực hiện truyền tải dữ liệu định danh sinh viên:
 - Client (Node gửi):** Đóng gói thông tin Mã số sinh viên (MSSV) của thành viên thực hiện vào gói tin Vendor Model và gửi lên mạng Mesh.
 - Server (Node nhận):** Lắng nghe, bắt gói tin từ Client, giải mã payload và hiển thị thông tin MSSV nhận được lên giao diện dòng lệnh (Console/Log) để kiểm tra tính toàn vẹn dữ liệu.

2. Phân tích và Thiết kế hệ thống

2.1 Cấu hình Gói tin (Data Structure)

Để truyền tải chuỗi ký tự MSSV, hệ thống sử dụng **Vendor Model** với cấu trúc dữ liệu tùy chỉnh. Do MSSV là chuỗi ký tự cố định (8 ký tự/MSSV), ta cần thiết lập độ dài gói tin (DATA_LENGTH) phù hợp với số lượng byte cần truyền.

Trong phạm vi bài lab này, gói tin được định nghĩa trong header `my_model_def.h` như sau:

- Vendor ID & Client ID:** Sử dụng ID riêng `0x1221` và `0x2222` để tránh xung đột với các model chuẩn (SIG Models).
- Opcode:** Sử dụng mã `0x1` để định danh loại tin nhắn truyền dữ liệu.
- Payload:** Mảng byte chứa mã ASCII của các ký tự trong MSSV.

2.2 Xử lý tại phía Client (Transmitter)

Thay vì gửi từng byte rời rạc, Client sẽ thực hiện quy trình đóng gói dữ liệu (Data Packing) trước khi phát tán (Publish):

- Khởi tạo bộ đệm:** Tạo một mảng data có kích thước cố định `DATA_LENGTH`.
- Sao chép dữ liệu:** Sử dụng hàm `memcpy()` để sao chép chuỗi ký tự MSSV vào bộ đệm này. Việc sử dụng `memcpy` giúp đảm bảo dữ liệu được copy chính xác theo từng block bộ nhớ.
- Gửi gói tin:** Gọi API `sl_btmesh_vendor_model_set_publication` để đẩy dữ liệu xuống stack Bluetooth Mesh và phát đi.

Đoạn mã hiện thực logic đóng gói tại Client:

```
1 // Hàm chuẩn bị và gửi dữ liệu MSSV
2 static void send_student_id(void) {
3     // Định nghĩa MSSV cần gửi (Ví dụ cho sinh viên 22207056)
4     const char *mssv_str = "22207056";
5
6     // Xóa bộ đệm để tránh dữ liệu rác
7     memset(sensor_data, 0, DATA_LENGTH);
8
9     // Sao chép chuỗi MSSV vào payload
10    // Sao chép 8 bytes tương ứng với 8 ký tự
11    memcpy(&sensor_data[0], mssv_str, 8);
12 }
```

```
13 // Ghi log để debug
14 app_log("Publishing MSSV: %s\r\n", mssv_str);
15
16 // Thực hiện gửi gói tin qua Vendor Model
17 sl_btmesh_vendor_model_set_publication(
18     my_model.elem_index,
19     my_model.vendor_id,
20     my_model.model_id,
21     SENSOR_STATUS_OPCODE, // Opcode 0x1
22     1,                      // Final
23     DATA_LENGTH,          // Độ dài dữ liệu
24     sensor_data             // Con trỏ dữ liệu
25 );
26
27 sl_btmesh_vendor_model_publish(
28     my_model.elem_index,
29     my_model.vendor_id,
30     my_model.model_id
31 );
32 }
```

2.3 Xử lý tại phía Server (Receiver)

Tại Server, sự kiện `sl_btmesh_evt_vendor_model_receive_id` đóng vai trò là trình phục vụ ngắt (Interrupt Service Routine) cho các gói tin đến. Quy trình xử lý gồm:

1. **Lọc gói tin:** Kiểm tra Opcode và độ dài payload. len để đảm bảo gói tin nhận được đúng là gói tin chứa MSSV (Opcode 0x1).
2. **Giải mã (Parsing):** Dữ liệu nhận được đang ở dạng mảng byte thô (Raw bytes). Hệ thống cần trích xuất các byte này và gán vào một chuỗi ký tự (String) kết thúc bằng ký tự NULL (\0) để có thể hiển thị đúng định dạng văn bản.
3. **Hiển thị:** Sử dụng hàm `app_log()` để in kết quả ra Terminal.

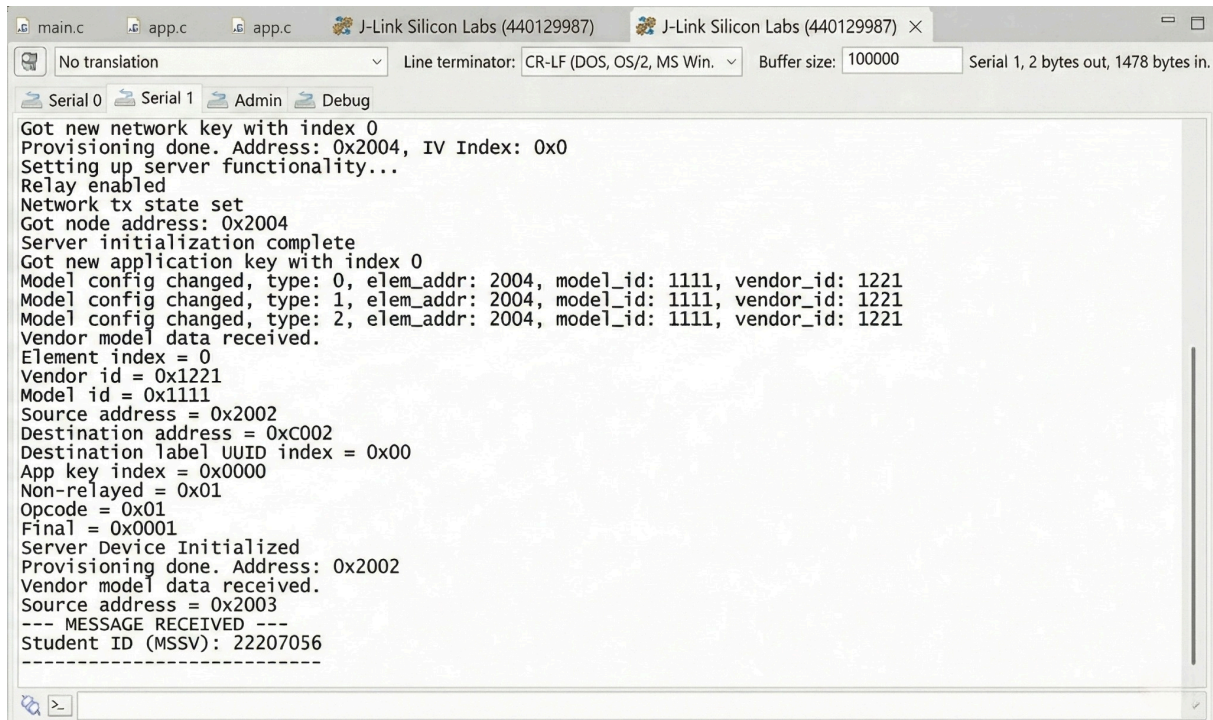
Đoạn mã hiện thực logic giải mã tại Server:

```
1 // Hàm xử lý sự kiện nhận gói tin Vendor Model
2 void sl_btmesh_on_event(sl_btmesh_msg_t *evt)
3 {
4     switch (SL_BT_MSG_ID(evt->header)) {
5         case sl_btmesh_evt_vendor_model_receive_id: {
6             sl_btmesh_evt_vendor_model_receive_t *rx_evt =
7                 (sl_btmesh_evt_vendor_model_receive_t *)&evt->data;
8
9             // --- PHÂN 1: IN THÔNG TIN HEADER GÓI TIN ---
10             app_log("Vendor model data received.\r\n\t"
11                 "Element index = %d\r\n\t"
12                 "Vendor id = 0x%04X\r\n\t"
13                 "Model id = 0x%04X\r\n\t"
```

```
13         "Source address = 0x%04X\r\n\t"
14         "Opcode = 0x%02X\r\n",
15         rx_evt->elem_index,
16         rx_evt->vendor_id,
17         rx_evt->model_id,
18         rx_evt->source_address,
19         rx_evt->opcode);
20
21     // --- PHÂN 2: GIẢI MÃ PAYLOAD ---
22     switch (rx_evt->opcode) {
23     case SENSOR_STATUS_OPCODE: // 0x1
24         // Đảm bảo gói tin có dữ liệu
25         if (rx_evt->payload.len > 0) {
26             char str_buffer[16];
27
28             app_log("--- MESSAGE RECEIVED ---\r\n");
29
30             // Trích xuất 8 byte dữ liệu vào buffer chuỗi
31             memcpy(str_buffer, rx_evt->payload.data, 8);
32             str_buffer[8] = '\0'; // Kết thúc chuỗi
33
34             app_log("Student ID (MSSV): %s\r\n", str_buffer);
35             app_log("-----\r\n");
36         }
37         break;
38     }
39     break;
40 }
41 }
42 }
```

3. Kết quả thực hiện

Hệ thống hoạt động ổn định, Server nhận được chính xác chuỗi ký tự được gửi từ Client.



```
main.c app.c app.c J-Link Silicon Labs (440129987) J-Link Silicon Labs (440129987) ×
No translation Line terminator: CR-LF (DOS, OS/2, MS Win. Buffer size: 100000 Serial 1, 2 bytes out, 1478 bytes in.
Serial 0 Serial 1 Admin Debug
Got new network key with index 0
Provisioning done. Address: 0x2004, IV Index: 0x0
Setting up server functionality...
Relay enabled
Network tx state set
Got node address: 0x2004
Server initialization complete
Got new application key with index 0
Model config changed, type: 0, elem_addr: 2004, model_id: 1111, vendor_id: 1221
Model config changed, type: 1, elem_addr: 2004, model_id: 1111, vendor_id: 1221
Model config changed, type: 2, elem_addr: 2004, model_id: 1111, vendor_id: 1221
Vendor model data received.
Element index = 0
Vendor id = 0x1221
Model id = 0x1111
Source address = 0x2002
Destination address = 0xc002
Destination label UUID index = 0x00
App key index = 0x0000
Non-relayed = 0x01
Opcode = 0x01
Final = 0x0001
Server Device Initialized
Provisioning done. Address: 0x2002
Vendor model data received.
Source address = 0x2003
--- MESSAGE RECEIVED ---
Student ID (MSSV): 22207056
-----
```

Hình 1: Console của Server - Hệ thống nhận và giải mã thành công MSSV (22207056) từ Client

BÀI 2: ỨNG DỤNG MESH - TRUYỀN TẢI TRẠNG THÁI LED VÀ THỜI GIAN HOẠT ĐỘNG

1. Yêu cầu bài toán

Xây dựng mạng Bluetooth Mesh gồm Client và Server với chức năng cụ thể:

- **Client:**

- Nút nhấn 0 (Button 0): Gửi thời gian hoạt động (Uptime) của thiết bị lên Server.
- Nút nhấn 1 (Button 1): Gửi trạng thái của 2 LED. Hai LED này hoạt động như một bộ đếm nhị phân 2-bit, tự động tăng giá trị sau mỗi 5 giây.

- **Server:**

- Nhận gói tin, giải mã dữ liệu dựa trên header và hiển thị thông tin tương ứng (thời gian hoặc trạng thái LED) lên màn hình LCD.

2. Phân tích và Thiết kế hệ thống

2.1 Cấu hình Model và Dữ liệu (Header Definition)

Để đảm bảo Server phân biệt được loại dữ liệu (Thời gian hay LED), cần định nghĩa cấu trúc gói tin (Payload) trong file header (my_model_def.h).

Hệ thống sử dụng cơ chế **Vendor Model** của Silicon Labs với VENDOR_ID và CLIENT_ID riêng biệt.

- **Độ dài dữ liệu (Data Length):**

- Thiết lập kích thước cố định là 8 bytes (DATA_LENGTH = 8).
- Kích thước này đủ để chứa 1 byte Header định danh và phần dữ liệu thực tế (4 bytes cho thời gian hoặc 1 byte cho trạng thái LED).

- **Cơ chế Header:** Quy ước byte đầu tiên (data[0]) của payload làm định danh để Server phân loại dữ liệu:

- Header 0x01: Dữ liệu đi kèm là thời gian hoạt động (Device Uptime).
- Header 0x02: Dữ liệu đi kèm là trạng thái của 2 LED.

```
1 // my_model_def.h
2 #define VENDOR_ID          0x1221 // ID Nhà cung cấp (tùy chỉnh)
3 #define MY_VENDOR_CLIENT_ID 0x2222 // ID Client
4 #define DATA_LENGTH       8      // Độ dài gói tin
5 #define SENSOR_STATUS_OPCODE 0x1  // Opcode cho bản tin trạng thái
```

2.2 Xử lý tại phía Client (Node Gửi)

Quy trình xử lý tại Client được chia thành hai luồng chính: cập nhật trạng thái LED theo thời gian thực và xử lý sự kiện nút nhấn để gửi dữ liệu.

a. Cơ chế Bộ đếm LED (Binary Counter)

Sử dụng một `sl_sleeptimer` hoạt động định kỳ (periodic) với chu kỳ 5000ms (5 giây). Trong hàm callback của timer, biến đếm trạng thái `led_counter_state` sẽ tự động tăng giá trị.

Trạng thái logic của LED0 và LED1 được tách ra từ biến đếm bằng các phép toán dịch bit (`>>`) và AND logic (`& 0x01`). Điều này giúp mô phỏng hoạt động đếm nhị phân 2-bit (00 -> 01 -> 10 -> 11).

```
1 // Hàm callback của Timer (chu kỳ 5s)
```



```
2 static void led_timer_callback(sl_sleeptimer_timer_handle_t* handle, void* data)
3 {
4     // Tách bit để điều khiển LED như bộ đếm nhị phân
5     // Bit 0 điều khiển LED0, Bit 1 điều khiển LED1
6     uint8_t bit_low = led_counter_state & 0x01;
7     uint8_t bit_high = (led_counter_state >> 1) & 0x01;
8
9     // Cập nhật trạng thái phân cứng (GPIO)
10    if (bit_low) {
11        GPIO_PinOutSet(BSP_GPIO_LED0_PORT, BSP_GPIO_LED0_PIN);
12    } else {
13        GPIO_PinOutClear(BSP_GPIO_LED0_PORT, BSP_GPIO_LED0_PIN);
14    }
15
16    if (bit_high) {
17        GPIO_PinOutSet(BSP_GPIO_LED1_PORT, BSP_GPIO_LED1_PIN);
18    } else {
19        GPIO_PinOutClear(BSP_GPIO_LED1_PORT, BSP_GPIO_LED1_PIN);
20    }
21
22    // Tăng biến đếm cho lần kế tiếp (reset về 0 nếu vượt quá 3)
23    led_counter_state++;
24    if (led_counter_state > 3) led_counter_state = 0;
25 }
```

b. Cơ chế Gửi dữ liệu (Publish)

Khi người dùng nhấn nút, hàm xử lý ngắt sẽ đóng gói dữ liệu vào mảng `sensor_data` trước khi gọi hàm `sl_btmesh_vendor_model_set_publication` và `publish`.

- **Khi nhấn Button 0:**

- Header: 0x01.
- Payload: Giá trị thời gian uptime (lấy từ hệ thống) được sao chép vào 4 byte tiếp theo của mảng dữ liệu.

- **Khi nhấn Button 1:**

- Header: 0x02.
- Payload: Giá trị `led_counter_state` hiện tại được gán vào byte tiếp theo.

2.3 Xử lý tại phía Server (Node Nhận):

Tại Server, sự kiện `sl_btmesh_evt_vendor_model_receive_id` được kích hoạt khi có gói tin gửi đến địa chỉ unicast của Server. Quá trình xử lý diễn ra như sau:

a. Kiểm tra tính hợp lệ

Trước khi xử lý, hệ thống kiểm tra `vendor_id`, `model_id` và độ dài dữ liệu (`payload.len`) để đảm bảo gói tin đúng định dạng mong đợi và loại bỏ các gói tin rác.

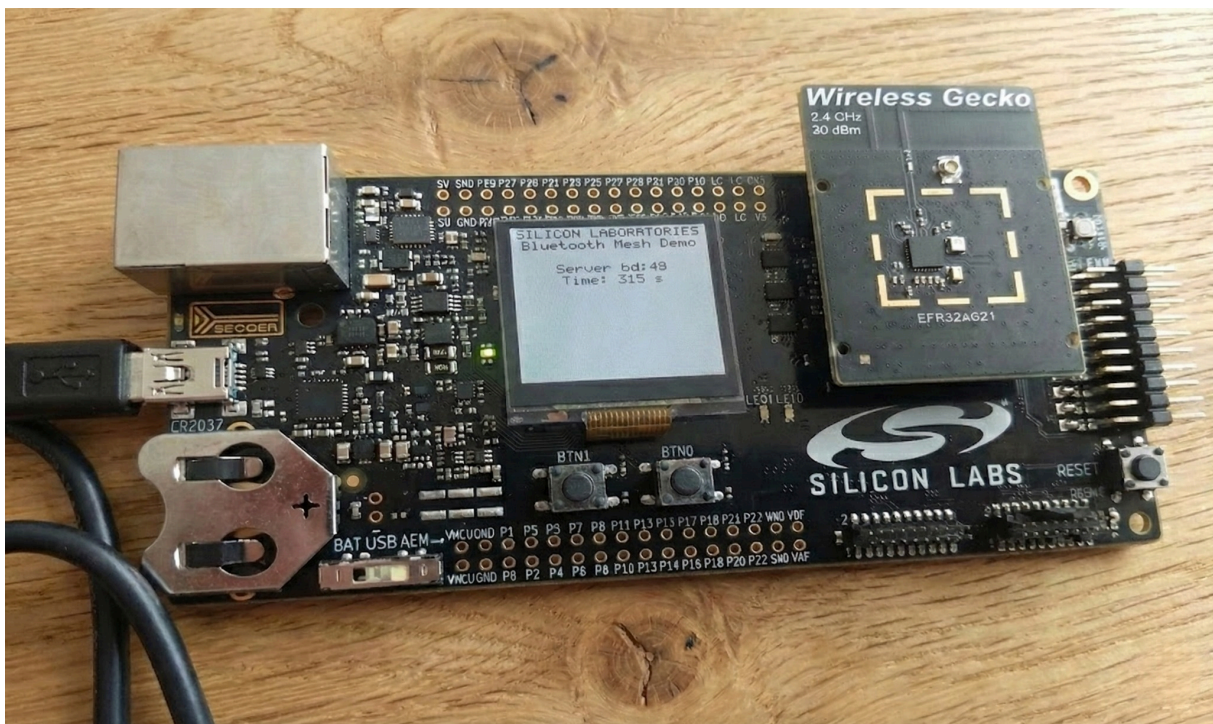
b. Phân loại và Giải mã (Parsing)

Hệ thống đọc byte Header (`data[0]`) để xác định định dạng dữ liệu cần xử lý và hiển thị lên LCD thông qua thư viện đồ họa (GLIB).

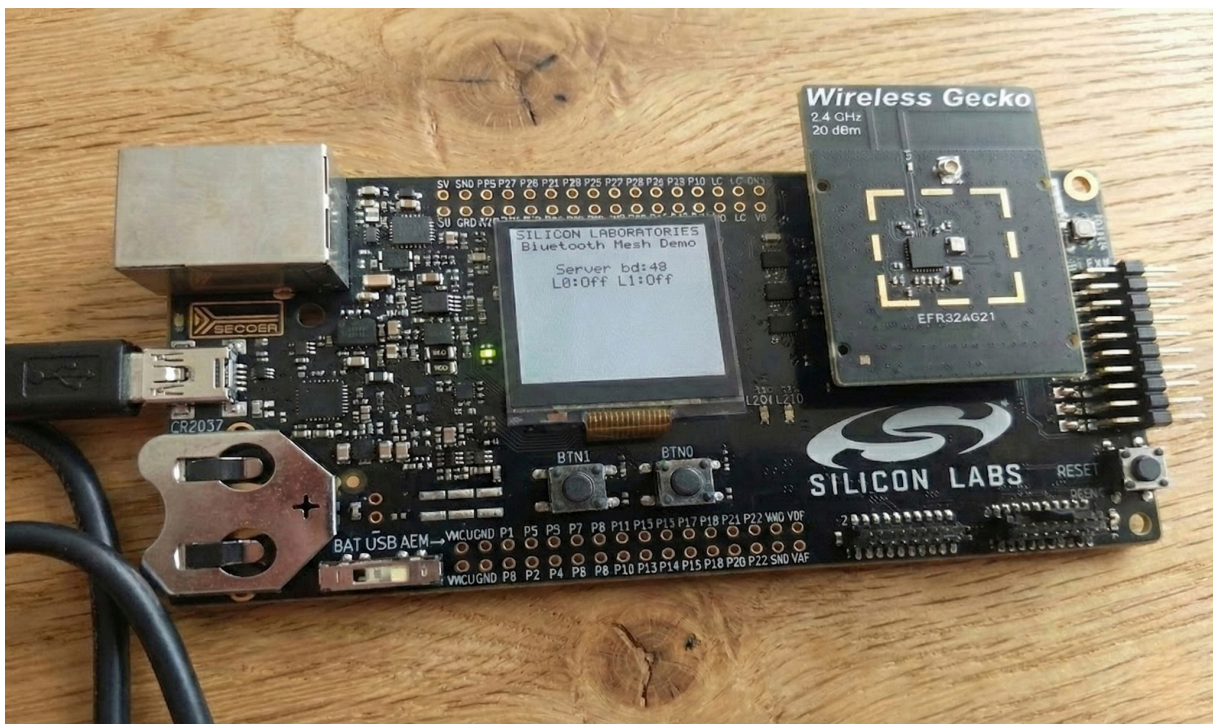
- **Trường hợp Header = 0x01 (Time):** Sử dụng hàm memcpy để trích xuất 4 byte dữ liệu thời gian từ payload ra biến uint32_t, sau đó định dạng chuỗi hiển thị.
- **Trường hợp Header = 0x02 (LED):** Đọc trạng thái LED từ byte dữ liệu thứ 2 (data[1]). Dựa vào giá trị bit để chuyển đổi sang chuỗi ký tự “On”/“Off” tương ứng.

```
1 // Đoạn mã xử lý trong sự kiện sl_btmesh_evt_vendor_model_receive_id
2 switch (rx_evt->opcode) {
3     case SENSOR_STATUS_OPCODE:
4         if (rx_evt->payload.len >= 2) {
5             uint8_t *data = rx_evt->payload.data;
6             uint8_t header = data[0];
7             char lcd_string[32];
8
9             // --- XỬ LÝ GÓI TIN THỜI GIAN (Header 0x01) ---
10            if (header == 0x01) {
11                uint32_t uptime_val = 0;
12                // Trích xuất 4 byte thời gian bắt đầu từ data[1]
13                memcpy(&uptime_val, &data[1], 4);
14
15                // Format chuỗi và hiển thị lên LCD
16                app_log("Type: DEVICE UPTIME\r\nVal: %lu seconds\r\n",
17                    uptime_val);
18                snprintf(lcd_string, sizeof(lcd_string), "Time: %lu s",
19                    uptime_val);
20                lcd_print(lcd_string, SL_BT_MESH_WSTK_LCD_ROW_STATUS_CFG_VAL);
21            }
22            // --- XỬ LÝ GÓI TIN LED (Header 0x02) ---
23            else if (header == 0x02) {
24                uint8_t led_state = data[1];
25
26                // Kiểm tra bit để xác định trạng thái On/Off
27                // Bit 0 -> LED0, Bit 1 -> LED1
28                char *str_led0 = (led_state & 0x01) ? "On" : "Off";
29                char *str_led1 = ((led_state >> 1) & 0x01) ? "On" : "Off";
30
31                app_log("Type: LED STATUS\r\nLED0: %s, LED1: %s\r\n", str_led0,
32                    str_led1);
33                snprintf(lcd_string, sizeof(lcd_string), "L0:%s L1:%s",
34                    str_led0, str_led1);
35                lcd_print(lcd_string, SL_BT_MESH_WSTK_LCD_ROW_STATUS_CFG_VAL);
36            }
37        }
38        break;
39    }
```

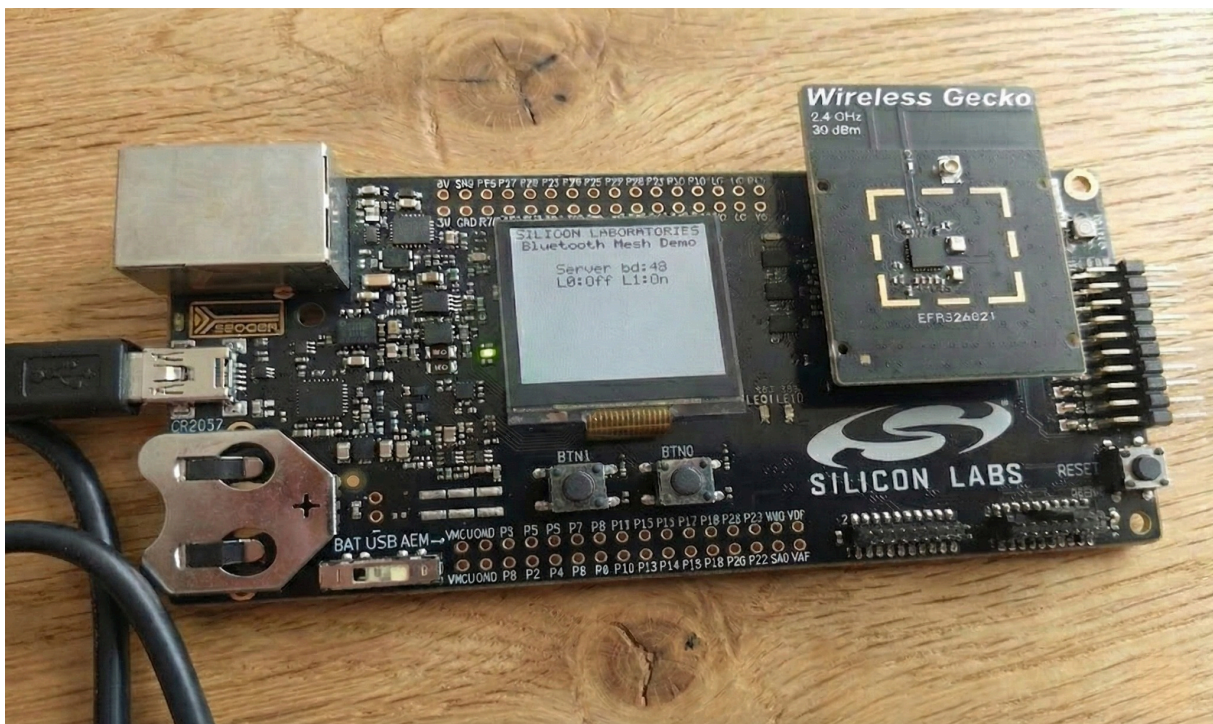
3. Kết quả thực hiện



Hình 2: Server - Màn hình LCD hiển thị thời gian hoạt động nhận được từ Client



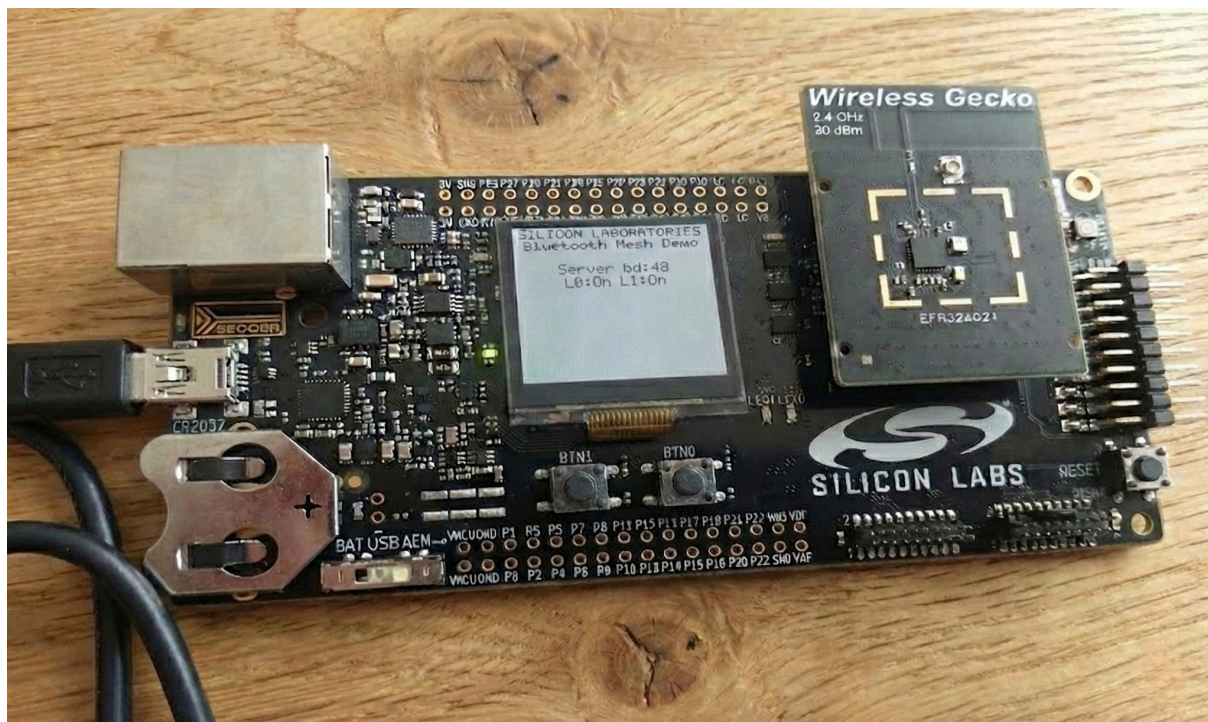
Hình 3: Server - Màn hình hiển thị trạng thái hai LED đều tắt (L0: Off, L1: Off)



Hình 4: Server cập nhật trạng thái LED 1 bật (L0: Off, L1: On)



Hình 5: Server cập nhật trạng thái LED 0 bật (L0: On, L1: Off)



Hình 6: Server hiển thị cả hai LED đều bật (L0: On, L1: On)